

AIKernel Core Specification: Chat-Turn HashChain Governance

Deterministic Chat-Turn Identity and Replayable Causal Trace Model for AIKernel.NET

Takuya Sogawa

2026-06-01

Author: Takuya Sogawa

Affiliation: AIKernel Project

ORCID: <https://orcid.org/0009-0009-7499-2595>

Version: v0.1.0

Date: 2026-06-01

DOI: 10.5281/zenodo.20471345

License: CC BY 4.0

Abstract

This specification defines Chat-Turn HashChain Governance as an AIKernel Core mechanism for converting chat-based interaction histories into causally linked, tamper-evident, and replayable semantic execution records.

In AIKernel, Phase-1 introduced ROM as a knowledge snapshot substrate, pre-inference admissibility governance, and trajectory governance. Phase-2 introduced semantic state, semantic transition, observability, and governed runtime theory. However, the causal identity of individual chat turns has not yet been defined as a first-class Core contract. This specification fills that gap by treating every chat turn as a canonicalized, hash-linked, optionally signed node in an append-only governance chain.

The main contribution of this document is to define a deterministic model for chat-turn canonicalization, hash-chain construction, signature verification, fail-closed validation, quarantine behavior, deterministic replay, VFS/ROM persistence, and cryptographic agility. The specification is algorithm-agile: AIKernel Core depends on provider interfaces such as `ISemanticHasher` and `IChatTurnSignatureProvider`, rather than on a fixed cryptographic algorithm.

The English manuscript is the canonical version. The Japanese manuscript is included as a companion translation.

1. Overview

AIKernel treats AI execution not as an unconstrained prompt-response exchange, but as a governed semantic trajectory. A trajectory must be observable, auditable, replayable, and bounded by contracts. In chat-based systems, however, the unit of interaction is usually a conversational turn, and ordinary chat logs do not by themselves guarantee causal identity, immutability, signature provenance, replay safety, or fork detection.

Chat-Turn HashChain Governance defines each chat turn as a canonicalized and hash-linked governance node. A turn is accepted into Kernel valid state only if its canonical representation, previous hash, current hash, optional signature, and governance policy checks all succeed.

The purpose of this specification is to transform nondeterministic LLM interaction history into a deterministic, verifiable, and replayable HashChain. In this model, a Semantic Trajectory is not merely a transcript. It is a causally linked execution record whose integrity can be checked at every turn.

Minimal Example of a Governed Chat-Turn HashChain

Role	PrevHash	CanonicalContent	Hash	State
User	0x00000000	T1	0x1619E3AA	Valid
Assistant	0x1619E3AA	T2	0x60192CAF	Valid
User	0x60192CAF	T3	0x07743DFF	Valid

Each turn incorporates the hash of the previous valid turn. A modification to any prior turn therefore invalidates the downstream chain.

2. Canonicalization Model

The prerequisite for a reliable hash chain is canonicalization. Two semantically identical turns must serialize to the same byte sequence, and two materially different turns must not collapse into the same canonical payload.

2.1 Canonical Form of a Chat Turn

A chat turn T_n is defined as a strict data structure containing at least the following fields.

Field	Meaning
Actor	Speaker or source, such as User, Assistant, System, or Tool.
Body	Text payload, tool result, or structured message content.
Timestamp	Strictly formatted timestamp, such as ISO 8601 UTC.
TurnKind	Turn category, such as message, tool call, tool result, or system event.
Metadata	Minimal metadata required for contractual correctness.

Metadata must be minimal. UI state, local cache identifiers, memory usage, elapsed processing time, and other nondeterministic runtime artifacts must not be included in the canonical hash payload.

2.2 Deterministic Serialization Rules

The canonicalization function f_{canon} serializes T_n according to deterministic rules.

1. Stable ordering: object properties are ordered lexicographically or by a specification-defined fixed order.

2. Whitespace normalization: unintentional leading/trailing whitespace and line ending differences are normalized, while semantically meaningful whitespace inside code blocks or quoted text is preserved.
3. Encoding: the final canonical payload is encoded using a specification-defined encoding, such as UTF-8.
4. Optional field handling: omitted optional fields, explicit null, and empty strings are not silently treated as equivalent.
5. Culture-invariant formatting: dates, numbers, decimal separators, and case transformations are formatted independently of locale.

2.3 Forbidden Fields

The canonical payload must not include fields that may change after hash computation or differ across machines.

- UI expansion state, scroll position, theme, or viewport state
- Processing duration or memory usage
- Temporary local file paths
- Nondeterministic random identifiers
- Provider-internal diagnostic ordering
- Cache keys not contractually defined
- Log ordering artifacts

The goal is to hash the governed semantic context, not incidental rendering or runtime state.

3. HashChain Definition

After canonicalization, each turn is hashed together with the previous valid hash. This forms a causal chain rather than a plain append-only list.

3.1 Formal Definition

The hash H_n of turn n is defined as:

$$H_n = Hash(Canonicalize(T_n) \parallel H_{n-1})$$

where $\parallel$ denotes byte-string concatenation. The initial value H_0 is the Kernel-defined Root Hash. A minimal implementation may use a fixed value such as 0x00000000, but production systems should derive the root from an explicit session boundary, Kernel identity, ROM root, or replay boundary.

3.2 Properties

Immutability If any previous turn T_{n-k} changes by even one bit, its hash changes. Every subsequent hash becomes invalid because it depends on the altered predecessor.

Causal Dependency H_n commits to the full causal conversation state up to turn n . The current Semantic Context is therefore defined not only by the latest message, but by the full valid chain leading to it.

Irreversibility A hash does not reveal the original message or context. It identifies causal state without exposing the content itself.

Compression of Semantic Entropy A large conversation history is compressed into a deterministic fixed-length state identifier. This identifier becomes the basis for replay, audit, fork detection, and recovery.

4. Signature Model

A hash chain detects modification, but it does not by itself prove who produced or approved a turn. For enterprise use, multi-provider execution, federation, and Phase-3 trust-layer extensions, selected turns or boundary turns should be signed.

4.1 Formal Definition

The signature $Sign_n$ for turn n is defined as:

$$Sign_n = Sign(H_n, K_{private})$$

where $K_{private}$ is the private key of the actor, provider, kernel, organization, or signing authority.

4.2 Behavior and Rules

Provider Identity A signature is associated with the provider, system, tool, kernel, organization, or actor that produced or approved the turn.

Verification Rules When accepting a turn, the Kernel verifies:

1. the canonicalized representation conforms to the specification;
2. PrevHash matches the current chain tail;
3. Hash matches the recomputed hash;
4. if present, the signature verifies under an authorized public key;
5. the algorithm tag is supported by the configured provider;
6. governance policy allows the turn to bind to the valid chain.

Fail-Closed Behavior If hash verification, signature verification, canonicalization, or algorithm routing fails, the turn must not bind to Kernel valid state. The Kernel fails closed and may place the turn into quarantine.

ReplayLog Integration Valid signed turns are persisted into ReplayLog-compatible storage, enabling later verification, audit, and deterministic reconstruction.

5. Governance Rules

AIKernel does not accept generated chat turns unconditionally. Each turn is checked by HashChain Governance before it becomes part of semantic state.

5.1 Validation Pipeline

1. Receive input turn T_n .
2. Compute $Canonicalize(T_n)$.
3. Combine it with the current tail hash H_{n-1} and recompute H_n .
4. Compare the recorded hash with the recomputed hash.
5. If a signature exists, verify it.
6. Apply governance policy.

7. Accept the turn into valid state only if all checks succeed.

5.2 On-the-Fly Verification

Verification is performed at each turn, not only at the end of a conversation. This prevents corrupted, injected, unsigned, or policy-denied turns from contaminating downstream context.

5.3 Quarantine Behavior and Failure Modes

A failed turn is placed in a quarantine region. Quarantined turns must not bind to Semantic Context, ReplayLog, Execution Pipeline, or Trajectory Evaluation.

Failure Mode	Description	Kernel Action
hash.mismatch	Recorded hash differs from recomputed hash.	Fail-Closed
prev_hash.mismatch	PrevHash does not match current tail.	Fail-Closed
signature.invalid	Signature cannot be verified.	Fail-Closed
algorithm.unsupported	Algorithm tag has no configured provider.	Fail-Closed
canonicalization.invalid	Canonicalization rules are violated.	Reject
policy.denied	Governance policy rejects the turn.	Quarantine
injection.detected	Prompt injection or unsafe payload is detected.	Quarantine

5.4 Recovery Semantics

When execution stops fail-closed, the Kernel can roll back to the last verified hash H_{n-1} . Execution may restart from that state, or an administrator may inspect quarantined turns before any rebinding operation.

6. Deterministic Replay

A protected chat-turn hash chain forms the basis of ReplayLog v2.

6.1 Rollback to Any Valid Turn

Given a valid node hash H_k , the Kernel can reconstruct the context up to that turn by replaying only verified chain nodes.

6.2 Reconstructing Semantic State

Replay reconstructs semantic state from the accepted chain, not from the nondeterministic behavior of a future LLM call. This separates historical state reconstruction from model stochasticity.

6.3 Bounded Trajectory Reconstruction

A semantic trajectory becomes a bounded and verifiable transition sequence. Every accepted state transition can be traced to a hash-linked, policy-validated, and optionally signed turn.

7. Runtime Interfaces

The following interfaces define a minimal AIKernel.NET implementation boundary.

namespace AIKernel.Abstractions.Governance.ChatChain;

```
/// <summary>
/// Canonicalizes a chat turn into a deterministic byte or string representation.
/// </summary>
public interface IChatTurnCanonicalizer
{
    string Canonicalize(IChatTurn turn);
}
```

```
/// <summary>
/// Computes the current state hash from canonical content and previous hash.
/// </summary>
public interface ISemanticHasher
{
    string ComputeHash(string canonicalContent, string previousHash);
}
```

```
/// <summary>
/// Provides cryptographic signatures for chat-turn hashes.
/// </summary>
public interface IChatTurnSignatureProvider
{
    Task<string> SignAsync(
        string hash,
        CancellationToken cancellationToken = default);
    Task<bool> VerifyAsync(
        string hash,
        string signature,
        CancellationToken cancellationToken = default);
}
```

```
/// <summary>
/// Verifies causal identity, hash continuity, signatures, and policy acceptance.
/// </summary>
public interface IChatTurnChainVerifier
{
    Result VerifyChain(IEnumerable<IHashChainNode> turns);
    Result VerifyNextTurn(
        IHashChainNode nextTurn,
        string currentTailHash);
}
```

ISemanticHasher and IChatTurnSignatureProvider are deliberately algorithm-neutral. Concrete implementations may use SHA-256, SHA-3, Ed25519, ECDSA P-256, ML-DSA, or future algorithms as long as the configured policy permits them.

8. Storage Model

Hash-chain persistence is integrated with AIKernel VFS and ROM storage. The storage model is not merely a persistence detail; it is part of the governance boundary. A stored chat turn must

remain causally linked, verifiable, and replayable after it leaves memory.

8.1 VFS Integration and Append-Only Enforcement

Hash-chained chat records are persisted through the AIKernel VFS layer. The VFS provider responsible for chat-turn storage must reject destructive overwrite of existing accepted records. Only append operations that extend the current valid chain tail are permitted.

A conforming storage provider therefore enforces the following rules.

- Hash-chain nodes are serialized as JSON, ROM Markdown, or another canonical storage format.
- Each stored node must contain PrevHash, Hash, and optionally Signature.
- Existing accepted records must not be overwritten in place.
- A new turn may be appended only if PrevHash matches the current valid tail hash.
- VFS write operations should perform write-time verification before persistence.
- ROM/VFS metadata may include algorithm tags, provider identity, verification status, quarantine state, and replay boundaries.

In this model, append-only behavior is a governance property, not a storage convenience. If a VFS provider cannot enforce append-only persistence, the Kernel must treat that provider as unsuitable for trusted HashChain storage unless an external immutable log or equivalent enforcement layer is present.

8.2 ROM Identity Anchoring

When a governed conversation is closed, finalized, or exported as a knowledge asset, the entire valid chain may be frozen as a single immutable ROM identity. At that point, the chat history is no longer treated as a mutable transcript. It becomes a sealed knowledge snapshot with persistent provenance.

This anchoring step connects Chat-Turn HashChain Governance to the AIKernel ROM model:

- the chain root and final tail hash define the causal boundary;
- the canonicalized turn sequence defines the semantic content;
- the signature metadata defines provenance and trust context;
- the ROM identity freezes the conversation as an immutable knowledge snapshot.

The resulting asset can be referenced, replayed, audited, or cited without trusting a mutable chat log.

9. Security Considerations

9.1 Length-Extension Attack Resistance

Classical Merkle-Damgard hash functions such as SHA-256 require care when used in constructions that resemble secret-prefix MACs, because length-extension attacks can permit an attacker to derive a valid digest for extended data under certain misuse patterns. Chat-Turn HashChain Governance does not place secret keys directly inside the hash payload and does not use a raw hash as a secret-prefix authentication code.

The specification is therefore keyless at the hash-chain level: integrity is provided by canonical payload binding, previous-hash binding, and optional signatures through `IChatTurnSignatureProvider`. In addition, SHA-3 / Keccak-style sponge constructions are recommended for `ISemanticHasher` implementations where deployments want to avoid Merkle-Damgard length-extension classes of misuse.

This does not remove the need for collision resistance, canonicalization correctness, signature verification, and provider policy validation. It means that the hash-chain construction itself should not rely on a secret-prefix hash pattern.

9.2 Prompt Injection and Contamination Propagation

If an attacker injects a malicious instruction into the body of a chat turn, pre-inference governance must inspect the turn before it is accepted into the valid chain. A rejected or suspicious turn may be recorded in quarantine with its own causal metadata, but it must not become part of the trusted semantic context.

The CTHC structure makes contamination localizable. Because every accepted turn is bound to an index, timestamp, canonical payload, previous hash, and current hash, the Kernel can identify where a contaminated statement appeared and which downstream turns depended on it. This supports quarantine checkpoints and prevents untrusted body content from mutating system prompts, Kernel configuration, provider credentials, or other Core Substrate state.

9.3 Hash Collision Resistance

Hash algorithms must provide sufficient collision resistance for the intended threat model. SHA-256 is a reasonable classical baseline; SHA-3 may be used where appropriate. The canonicalization process must not collapse semantically distinct turns into identical hash payloads.

9.4 Signature Forgery and Replay Attacks

Signature algorithms must be selected according to the deployment policy. Incorporating H_{n-1} into H_n makes it structurally difficult to transplant an old signed turn into another chain position without detection.

9.5 Chain-Break Detection

If PrevHash does not match the current tail, the chain has broken. The Kernel must stop fail-closed and raise a security alert or recovery workflow.

9.6 Cryptographic Agility

AIKernel hash-chain and signature governance is designed to avoid dependence on any single cryptographic algorithm. This is a deliberate architectural choice. Post-quantum cryptography is progressing, but native runtime support, side-channel hardening, interoperability, and performance profiles are still evolving in many deployment environments.

Algorithm Independence through Provider Abstraction Hashing and signing are abstracted through provider interfaces such as:

- ISemanticHasher
- IChatTurnSignatureProvider

AIKernel Core therefore does not need to know whether a specific deployment uses classical cryptography or post-quantum cryptography. Algorithm replacement can be achieved by dependency injection and policy configuration.

Algorithm-Tagged Payload Format Signature payloads stored in ROM, VFS, or ReplayLog must carry an explicit algorithm tag.

<algorithm-id>:<base64-signature>

Examples:

ed25519:AbCdEf...

ml-dsa-65:ZyXwVu...

This enables backward compatibility, gradual post-quantum migration, provider routing, and future federation.

v0.1.0 Implementation Guidance For v0.1.0, classical algorithms such as Ed25519 and ECDSA P-256 are recommended as practical baselines where supported by the target platform and library stack. They are mature, widely implemented, and sufficient for validating fail-closed governance behavior.

Post-quantum algorithms such as ML-DSA should be introduced by replacing IChatTurnSignatureProvider implementations when operational maturity, runtime support, side-channel considerations, and deployment policy allow.

This specification therefore treats cryptographic agility as a Core invariant: the chain format must preserve algorithm identity, and Core governance must route verification through provider contracts rather than hard-coded algorithms.

10. Minimal Complete Example

10.1 Scenario and Canonicalization

The following example shows a three-turn governed conversation. The hash values are illustrative shortened values for readability. A real implementation must use the full output of the configured ISemanticHasher.

Turn 1: User Init

- Input actor: User
- Body: Hello, AIKernel.
- Timestamp: 2026-05-31T10:00:00.000Z
- Previous hash H_0 : 0x00000000
- Canonicalized data C_1 :
actor:User
body:Hello, AIKernel.
timestamp:2026-05-31T10:00:00.000Z
- Computed hash H_1 : $Hash(C_1 || "0x00000000") \rightarrow 0x1619E3AA$

Turn 2: Assistant Response

- Input actor: Assistant
- Body: System initialized.
- Timestamp: 2026-05-31T10:00:02.000Z
- Previous hash H_1 : 0x1619E3AA

- Canonicalized data C_2 :
actor:Assistant
body:System initialized.
timestamp:2026-05-31T10:00:02.000Z
- Computed hash H_2 : $Hash(C_2 || "0x1619E3AA") \rightarrow 0x60192CAF$

Turn 3: User Next Command

- Input actor: User
- Body: Load ROM.
- Timestamp: 2026-05-31T10:00:05.000Z
- Previous hash H_2 : 0x60192CAF
- Canonicalized data C_3 :
actor:User
body:Load ROM.
timestamp:2026-05-31T10:00:05.000Z
- Computed hash H_3 : $Hash(C_3 || "0x60192CAF") \rightarrow 0x07743DFF$

10.2 Verification Process

When the runtime verifies the integrity of the history, IChatTurnChainVerifier binds the recorded predecessor H_2 (0x60192CAF) to the canonicalized payload C_3 and independently recomputes the next hash. If the recomputed value matches the recorded H_3 (0x07743DFF), and if all governance and signature policies accept the turn, the causal relation from Turn 2 to Turn 3 is valid under the configured cryptographic assumptions.

Repeating this procedure from the genesis hash to the final tail hash verifies the continuity of the entire conversation. Any modification to an earlier canonical payload changes its hash and invalidates all downstream links.

11. Future Work

Future extensions include:

- integration with Phase-3 trust and self-evolving governance;
- multi-agent chain merging using DAG-style causal structures;
- cross-provider signature federation;
- long-term key rotation and algorithm migration;
- signed replay boundary manifests;
- hardware-backed signing and key management;
- PQC provider evaluation once runtime support and operational practices mature.

12. Non-Claims and Limitations

This specification does not claim that a hash chain makes LLM output deterministic. It only makes the accepted turn sequence verifiable and replayable.

It does not mandate a specific canonicalization standard, although implementations may align with existing canonical JSON approaches.

It does not require post-quantum cryptography in v0.1.0. Instead, it requires algorithm agility so that future cryptographic providers can be introduced without changing Core semantics.

It does not make quarantined turns trustworthy. Quarantine only prevents invalid turns from contaminating the valid chain.

13. Conclusion

Chat-Turn HashChain Governance turns chat history into a causally linked, cryptographically verifiable, and replayable semantic execution record. By combining deterministic canonicalization, previous-hash binding, optional signatures, fail-closed validation, quarantine, replay, and cryptographic agility, AIKernel gains a Core mechanism for protecting the integrity of semantic trajectories.

This specification provides the minimal contract boundary required to treat chat turns not as disposable UI messages, but as governed execution nodes in a Semantic Context Operating System.

References

1. Rundgren, Anders, Bryce Jordan, and Samuel Erdtman. "JSON Canonicalization Scheme (JCS)." RFC 8785, 2020. DOI: 10.17487/RFC8785.
2. National Institute of Standards and Technology. Secure Hash Standard (SHS). FIPS 180-4, 2015. DOI: 10.6028/NIST.FIPS.180-4.
3. National Institute of Standards and Technology. SHA-3 Standard: Permutation-Based Hash and Extendable-Output Functions. FIPS 202, 2015. DOI: 10.6028/NIST.FIPS.202.
4. Josefsson, Simon, and Ilari Liusvaara. "Edwards-Curve Digital Signature Algorithm (EdDSA)." RFC 8032, 2017. DOI: 10.17487/RFC8032.
5. National Institute of Standards and Technology. Digital Signature Standard (DSS). FIPS 186-5, 2023. DOI: 10.6028/NIST.FIPS.186-5.
6. National Institute of Standards and Technology. Module-Lattice-Based Digital Signature Standard. FIPS 204, 2024. DOI: 10.6028/NIST.FIPS.204.
7. Sogawa, Takuya. "Interface-Led Architecture (ILA): A Software Development Methodology for the AI Era, Validated by the AIKernel Execution Model." Zenodo, 2026. DOI: 10.5281/zenodo.20290614.
8. Sogawa, Takuya. "Provider-Observer-Operator: A Role-Based Abstraction Discipline for Interface-Led Architecture." Zenodo, 2026. DOI: 10.5281/zenodo.20322690.
9. Sogawa, Takuya. "AIKernel Formal Foundations: Contract-Based Semantic Execution for Governed AI Systems." Zenodo, 2026. DOI: 10.5281/zenodo.20460322.